

LAB Work

RTL to GDSII flow for 8 bit ALU with layered
verification

Functionality

OP CODE	FUNCTIONALITY	OP CODE	FUNCTIONALITY
0	Addition	8	Bitwise Negation
1	Subtraction	9	Bitwise AND
2	Multiplication	10	Bitwise OR
3	Division	11	Bitwise XOR
4	Modulo Division	12	Left Shift
5	Logical AND	13	Right Shift
6	Logical OR	14	Increment
7	Logical Negation	15	Decrement

```
module alu (out, a, b, s, clk);
```

```
input clk;
```

```
input [7:0] a, b;
```

```
input [3:0] s;
```

```
output [15:0] out;
```

```
reg [15:0] out;
```

```
always @(posedge clk) begin
```

```
  case (s)
```

```
    4'b0000: out = a + b;    // 8-bit addition
```

```
    4'b0001: out = a - b;    // 8-bit subtraction
```

```
    4'b0010: out = a * b;    // 8-bit multiplication
```

```
    4'b0011: out = a / b;    // 8-bit division
```

```
    4'b0100: out = a % b;    // 8-bit modulo division
```

```
    4'b0101: out = a && b;    // 8-bit logical AND
```

```
    4'b0110: out = a || b;    // 8-bit logical OR
```

```
    4'b0111: out = !a;        // 8-bit logical negation
```

```
    4'b1000: out = ~a;        // 8-bit bitwise negation
```

```
    4'b1001: out = a & b;    // 8-bit bitwise AND
```

```
    4'b1010: out = a | b;    // 8-bit bitwise OR
```

```
    4'b1011: out = a ^ b;    // 8-bit bitwise XOR
```

```
    4'b1100: out = a << 1;    // left shift
```

```
    4'b1101: out = a >> 1;    // right shift
```

```
    4'b1110: out = a + 1;    // increment
```

```
    4'b1111: out = a - 1;    // decrement
```

```
  endcase
```

```
end
```

```
endmodule
```

Self checking ALU in flat testbench

```
module alustimulus;

    reg clk;
    reg [7:0] a, b;
    reg [3:0] s;
    wire [15:0] out;

    initial begin
        forever #5 clk = ~clk;
    end

    alu DUT (
        .clk (clk),
        .a (a),
        .b (b),
        .s (s),
        .out (out)
    );

    initial begin
        clk = 1'b0;
        a = $random;
        b = $random;
        s = 4'bx;

        #30
        s = 0;
        #10
        result_checker(s, a, b, out);

        #10
        s = 14;
        #10
        result_checker(s, a, b, out);

        #10
        s = $random;
        #10
        result_checker(s, a, b, out);

        $finish;
    end

    task result_checker;
        input [3:0] s;
        input [7:0] a, b;
        input [15:0] resulted_out;
        reg [15:0] expected_out;
        begin
            if (s == 0)    expected_out = a + b;
            else if (s == 1) expected_out = a - b;
            else if (s == 2) expected_out = a * b;
            else if (s == 3) expected_out = a / b;
            else if (s == 4) expected_out = a % b;
            else if (s == 5) expected_out = a && b;
            else if (s == 6) expected_out = a || b;
            else if (s == 7) expected_out = !a;
            else if (s == 8) expected_out = ~a;
            else if (s == 9) expected_out = a & b;
            else if (s == 10) expected_out = a | b;
            else if (s == 11) expected_out = a ^ b;
            else if (s == 12) expected_out = a << 1;
            else if (s == 13) expected_out = a >> 1;
            else if (s == 14) expected_out = a + 1;
            else if (s == 15) expected_out = a - 1;

            if (resulted_out == expected_out)
                $display("Passed : a=%d, b=%d, s=%d, resulted_out=%d, expected_out=%d",
                    a, b, s, resulted_out, expected_out);
            else
                $display("Failed : a=%d, b=%d, s=%d, resulted_out=%d, expected_out=%d",
                    a, b, s, resulted_out, expected_out);
        end
    endtask

    // Waveform dumping
    initial begin
        $dumpfile("aludump.vcd");
        $dumpvars;
    end

endmodule
```

```
xcelium> run
Passed : a= 36, b=129, s= 0, resulted_out= 165, expected_out= 165
Passed : a= 36, b=129, s=14, resulted_out= 37, expected_out= 37
Passed : a= 36, b=129, s= 9, resulted_out= 0, expected_out= 0
Simulation complete via $finish(1) at time 80 NS + 0
./testbench.sv:38 $finish;
xcelium> exit
```

EDA Playground

<https://www.edaplayground.com/x/jMLa>

From: 0ns

To: 80ns

Get Signals

Radix



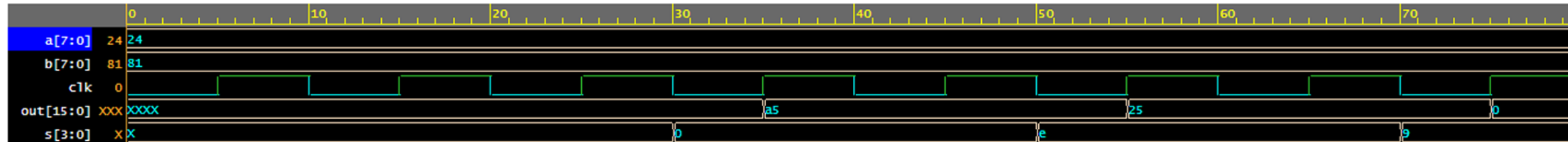
100%



0ns

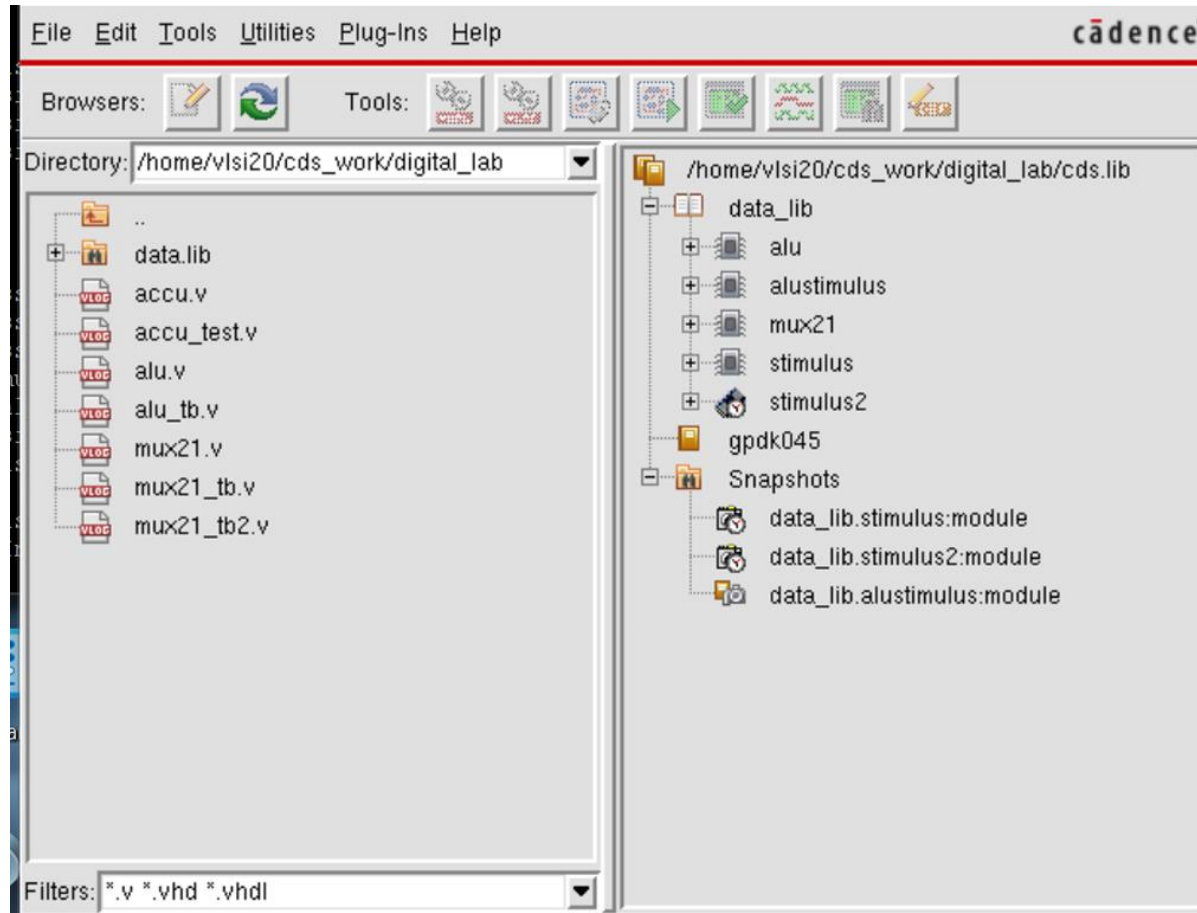


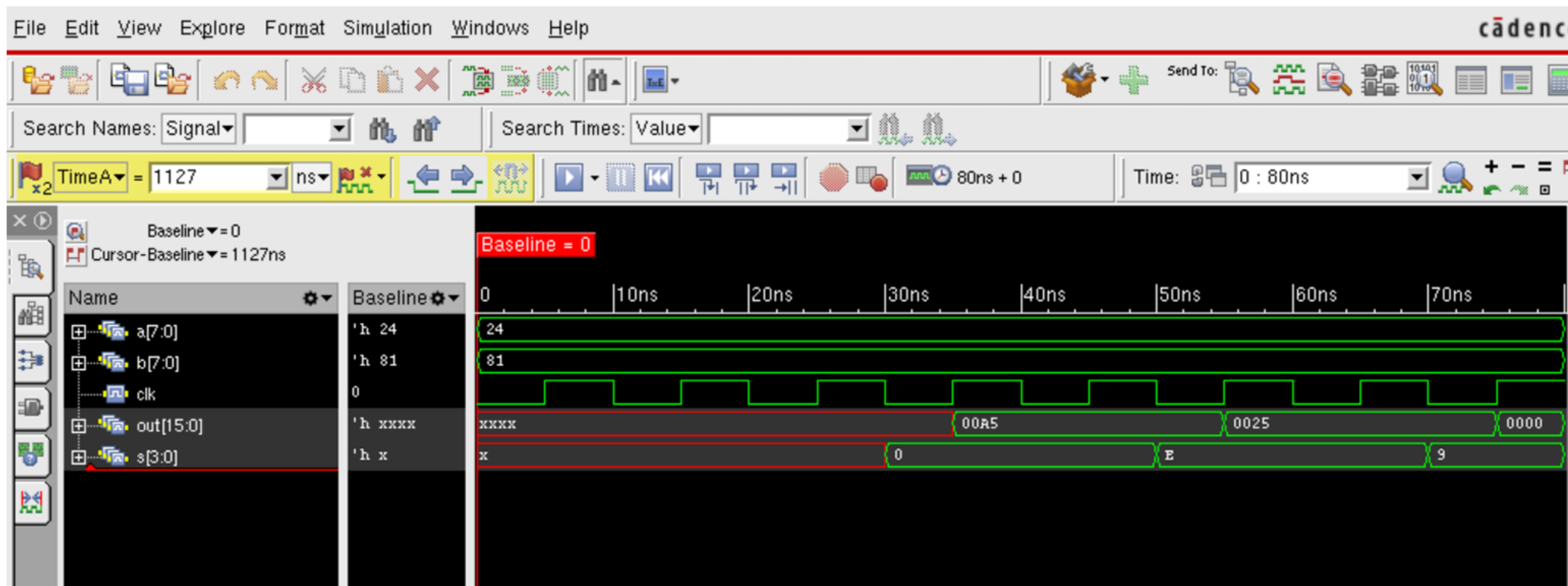
testbench/DUT/a[7:0]



Compile, elaborate and simulate using gui (nclaunch)

Input files





Layered verification implementation

- Limitation of flat testbenches:**

A flat testbench becomes very complex and difficult to maintain when it tries to generate all legal/illegal stimuli, handle multi-layer protocols, and inject errors using a single routine.

- Concept of layered testbench:**

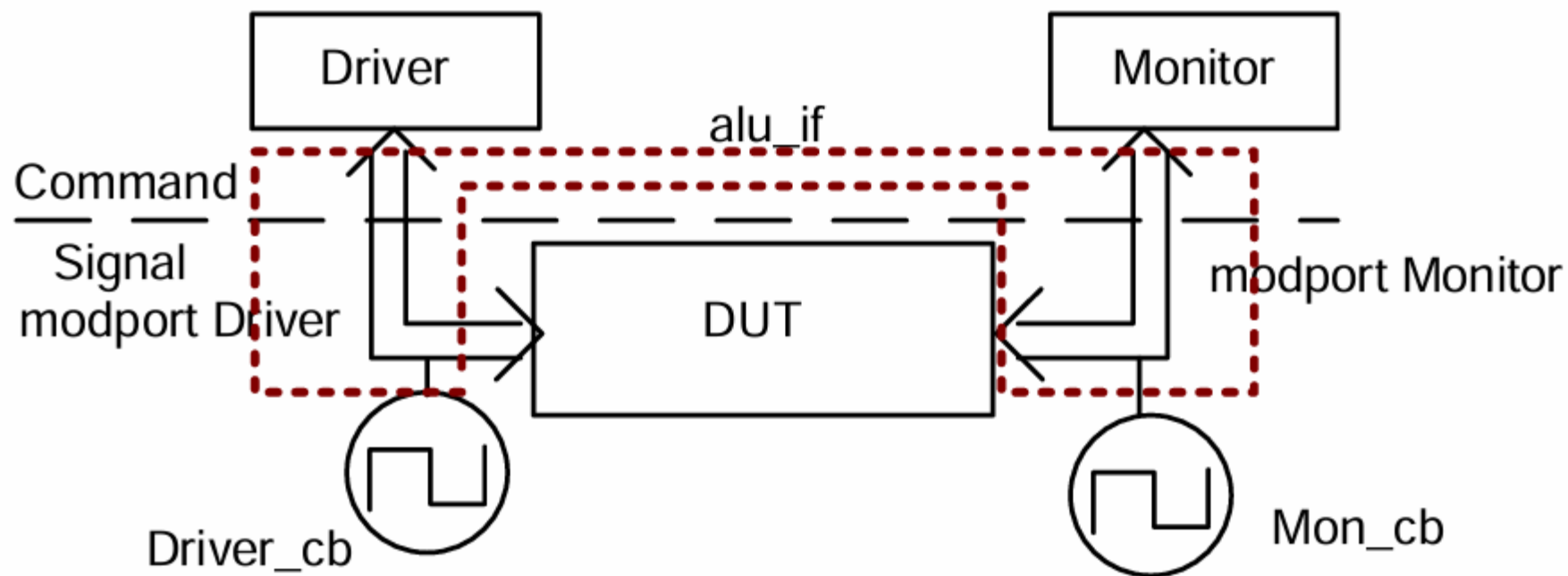
The layered testbench divides verification tasks into **smaller, independent components**, making development, debugging, and maintenance easier rather than more complex.

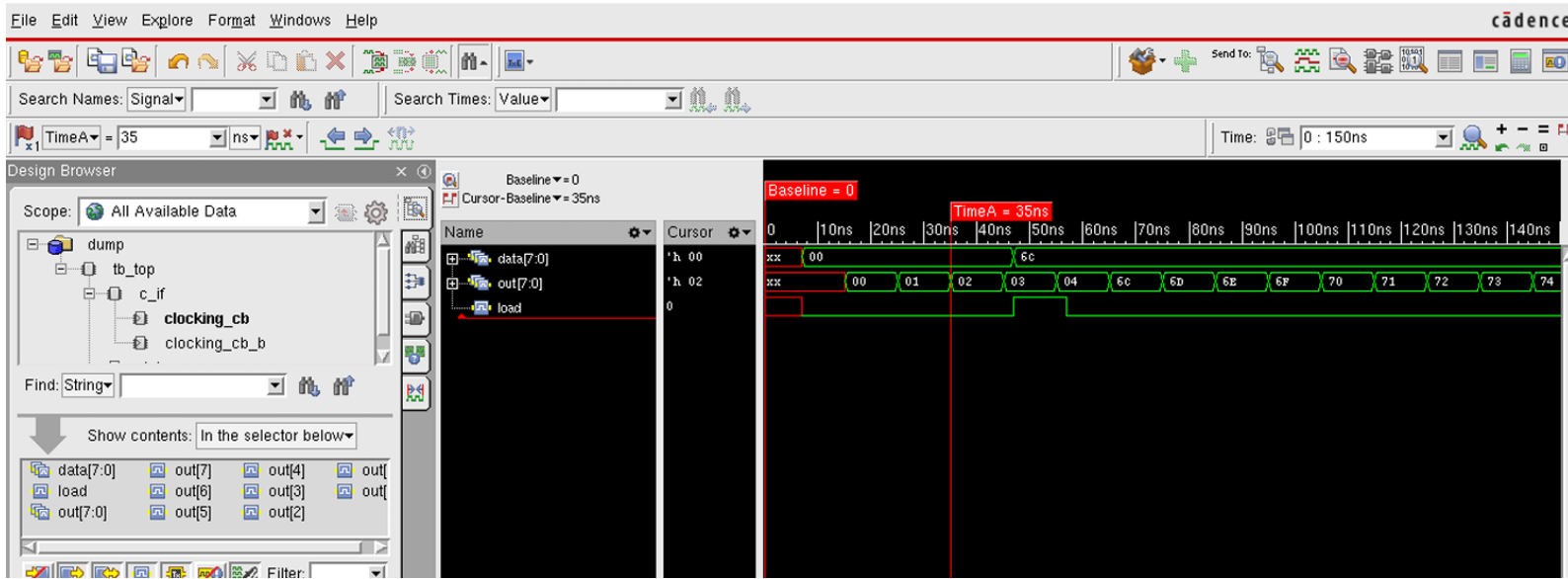
- Improved manageability:**

More organized and scalable.

- Limitation of Verilog:**

Traditional Verilog does not support OOP features such as classes, inheritance, and encapsulation, making it unsuitable for advanced layered verification environments.





```
ncsim> run
```

```
-----Scoreboard Test Starts-----
```

```
Passed : a= 0 b=128 s=15 Expected out=65535 Resulted out=65535
```

```
Passed : a= 8 b=235 s=15 Expected out= 7 Resulted out= 7
```

```
Passed : a= 19 b=137 s=15 Expected out= 18 Resulted out= 18
```

```
Passed : a= 97 b= 77 s=14 Expected out= 98 Resulted out= 98
```

```
Passed : a= 34 b= 91 s=15 Expected out= 33 Resulted out= 33
```

```
-----Scoreboard Test Ends-----
```

```
Simulation complete via $finish(1) at time 65 NS + 1
```

```
../testbench/environment.sv:39 $finish;
```

Synthesis

- Library used:

File Name	Uses
<i>slow_vdd1v0_basicCells.lib</i>	Standard cell library for slow corner operation (setup check)
<i>fast_vdd1v0_basicCells.lib</i>	Standard cell library for fast corner operation (hold check)
<i>gsclib045_tech.lef</i>	Process technology .lef file for 45nm tech node
<i>gsclib045_macro.lef</i>	Standard cell physical information for 45nm tech node

TCL

```
set TOP alu

set RTL_FILE "alu.v"

set OUT_DIR "./out"

set LIB_DIR "./library"

set TECH_DIR "./technology"
```

```
file mkdir $OUT_DIR

file mkdir "$OUT_DIR/reports"
```

```
set target_library [list "$LIB_DIR/slow_vdd1v0_basicCells.lib"
"$LIB_DIR/fast_vdd1v0_basicCells.lib"]

set link_library [list "$LIB_DIR/slow_vdd1v0_basicCells.lib"
"$LIB_DIR/fast_vdd1v0_basicCells.lib"]
```

```
set lef_files [list "$TECH_DIR/gsclib045_tech.lef"
"$TECH_DIR/gsclib045_macro.lef"]
```

```
# Constraints
```

```
set sdc_file "$OUT_DIR/alu.sdc"
```

```
read_verilog $RTL_FILE

link_design $TOP
```

```
create_clock -period 10 -name clk [get_ports clk]

create_reset -name rst_n [get_ports rst_n]
```

```
#Set library

read_liberty $target_library

read_lef $lef_files
```

```
source $sdc_file
```

```
# Elaborate

elaborate $TOP
```

```
# Synthesize Design
```

```
compile_ultra -uc -delay_mode_max -timing_derate 0.9
```

```
write_verilog "$OUT_DIR/alu_synthesized.v"
```

```
report_timing -max_paths 5 -sort_by slack >
"$OUT_DIR/reports/timing_report.txt"
```

```
report_area > "$OUT_DIR/reports/area_report.txt"
```

SDC

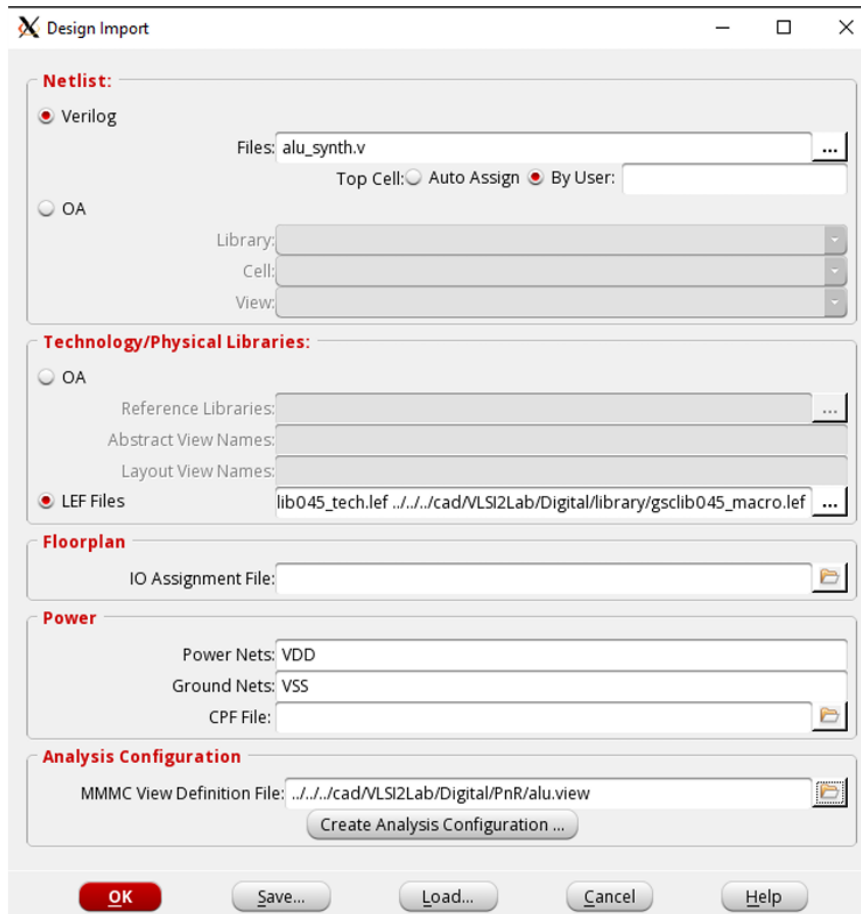
```
create_clock -period 10 -name clk [get_ports clk] # uncertainty
Define the clock period (10ns)
set_clock_uncertainty 0.1 -hold # Hold
set_input_delay -clock clk 2 -max [get_ports a] # uncertainty
Define input delay for port 'a'
set_input_delay -clock clk 2 -max [get_ports b] #
Define input delay for port 'b'
set_input_delay -clock clk 2 [get_ports a] # Input
delay for signal 'a'
set_output_delay -clock clk 2 [get_ports out] #
Define output delay for port 'out'
Output delay for 'out'
set_false_path -from [get_pins rst_n] -to [get_pins *]
set_max_delay -from [get_pins a] -to [get_pins out] 5ns
# Max path delay between 'a' and 'out'
set_clock_uncertainty 0.1 -setup # Setup
```

Area report

Instance	Cells	Cell Area	Net Area	Total Area	Wirebad	WL Flag
alu	2480	6400.87	0.00	6400.87	<none>	(D)
alu/add_15_30	32	162.79	0.00	162.79	<none>	(D)
alu/dec_sub_24_30_2	62	95.76	0.00	95.76	<none>	(D)
alu/inc_add_23_30_1	31	116.62	0.00	116.62	<none>	(D)
alu/mul_17_30	1774	4745.93	0.00	4745.93	<none>	(D)
alu/sub_16_30	97	206.23	0.00	206.23	<none>	(D)

Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
alu	2480	130.59	279037.49	101091.37	380128.86
alu/add_15_30	32	3.23	4968.29	798.19	5766.48
alu/dec_sub_24_30_2	62	2.47	885.79	85.64	971.43
alu/inc_add_23_30_1	31	2.68	684.90	104.62	789.53
alu/mul_17_30	1774	96.57	258679.84	91907.68	350587.52
alu/sub_16_30	97	4.56	5787.87	1478.46	7266.33

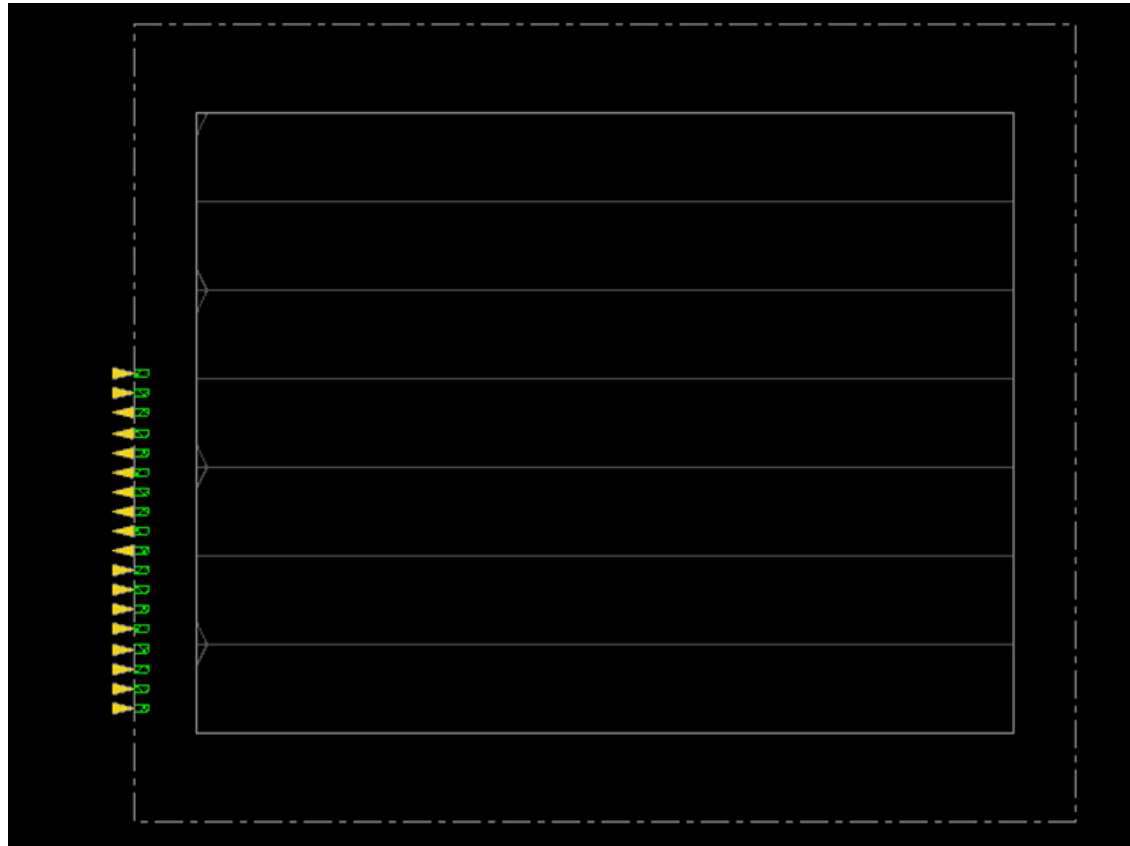
Physical Design : Cadence Innovus



Hier Cell - alu, 2480 LeafCells

- Terms (136)
 - out(output bus)
 - clk (input) - clk
 - en (input) - en
 - in_a(input bus)
 - in_b(input bus)
 - opcode(input bus)
 - rst_n (input) - rst_n
- Nets (785)
- StdCells (484)
 - g2 (NOR2BX1)
 - g9243 (NAND4XL)
 - g9244 (NAND4XL)
 - g9245 (NAND4XL)
 - g9246 (NAND4XL)
 - g9247 (NAND4XL)
 - g9248 (NAND4XL)
 - g9249 (NAND4XL)
 - g9250 (NAND4XL)
 - g9251 (NAND4XL)
 - g9266 (AOI221X1)
 - g9267 (AOI221X1)
 - g9268 (AOI221X1)
 - g9269 (AOI221X1)
 - g9270 (AOI221X1)
 - g9271 (AOI221X1)
 - g9272 (AOI221X1)

Floorplan and pin placement



Specify Floorplan

Basic Advanced

Design Dimensions

Specify By: ☒ Size ☐ Die/IO/Core Coordinates

☐ Core Size by: ☐ Aspect Ratio: Ratio (H/W): 594936709

☒ Core Utilization: 0.495479

☐ Cell Utilization: 0.495479

☒ Dimension: Width: 15.8

Height: 11.97

☒ Die Size by: Width: 18.2

Height: 15.39

Core Margins by: ☒ Core to IO Boundary

☐ Core to Die Boundary

Core to Left: 1.2 Core to Top: 1.71

Core to Right: 1.2 Core to Bottom: 1.71

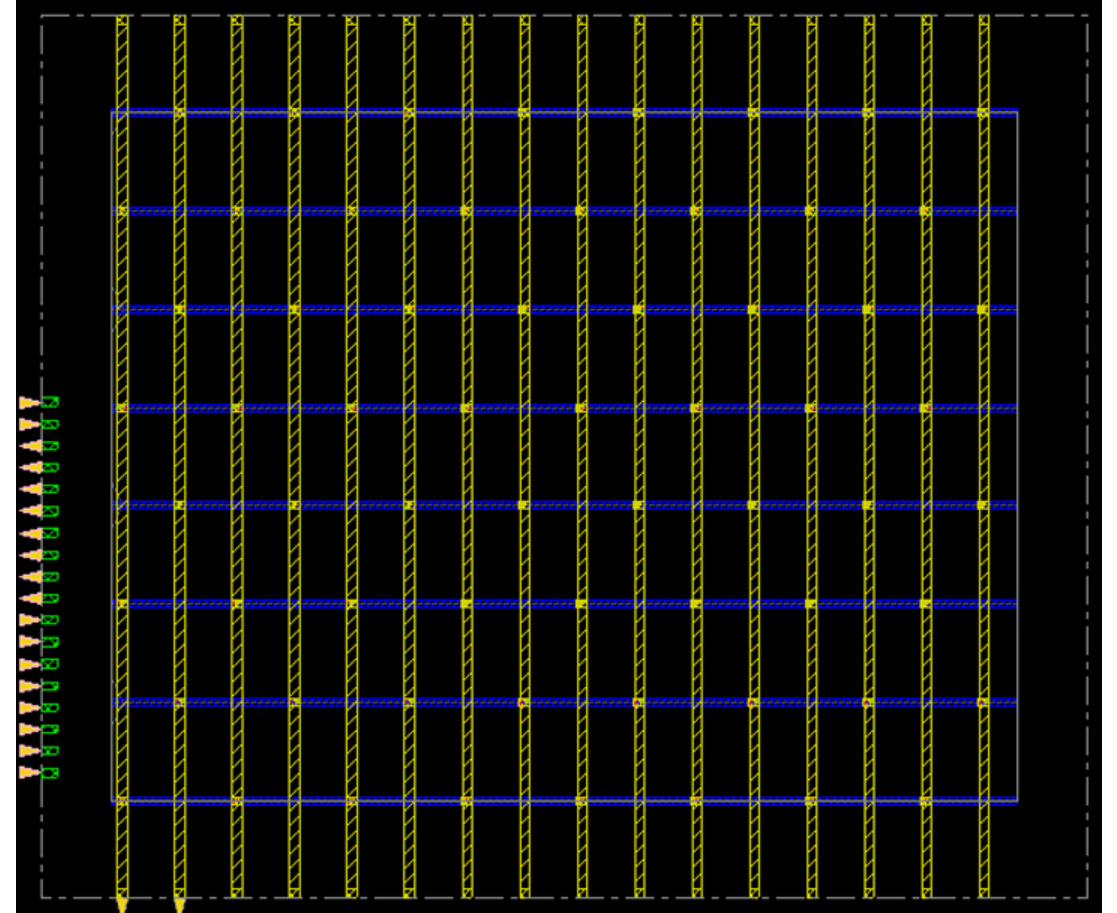
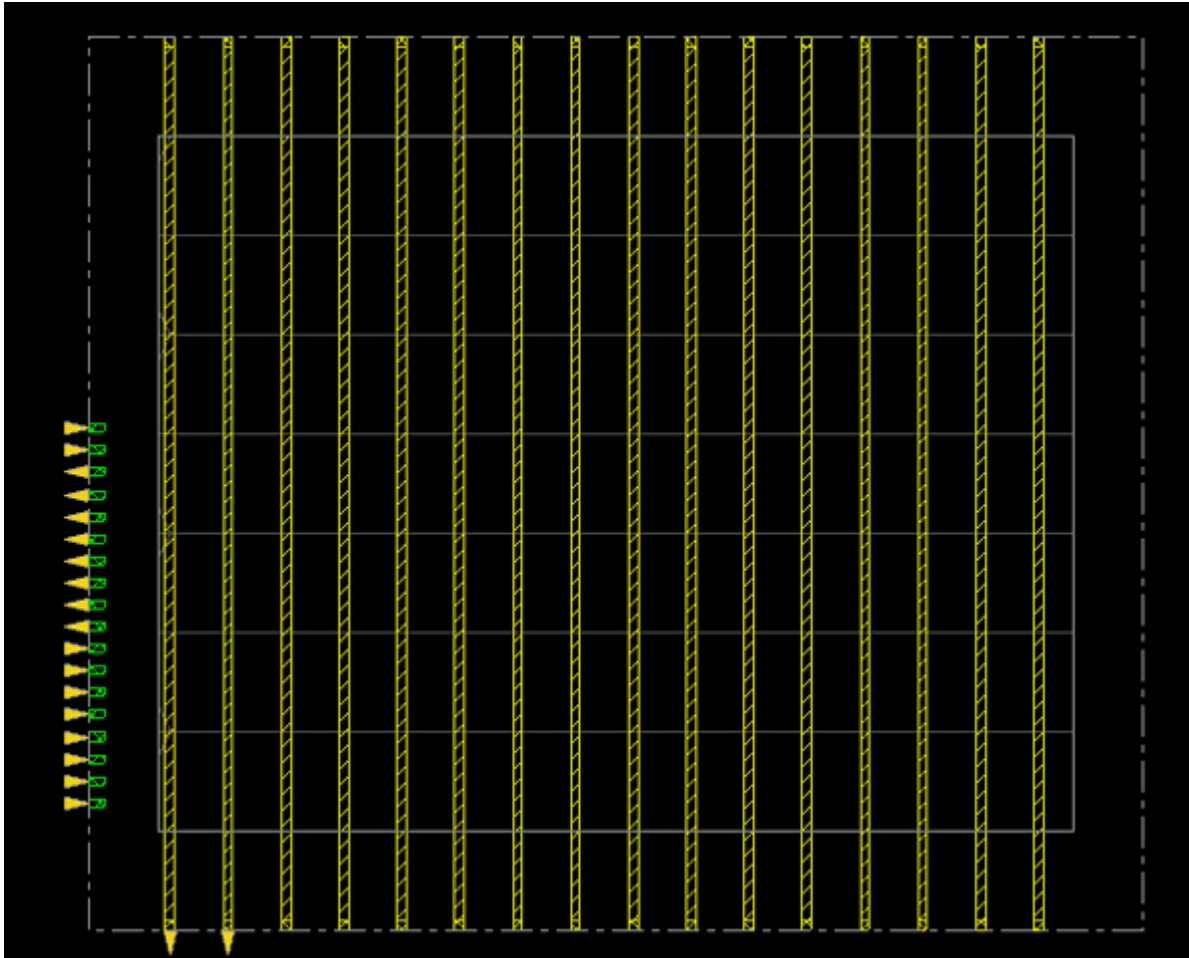
IO Box Calculation Use: ☐ Max IO Height ☒ Min IO Height

Floorplan Origin at: ☒ Lower Left Corner ☐ Center

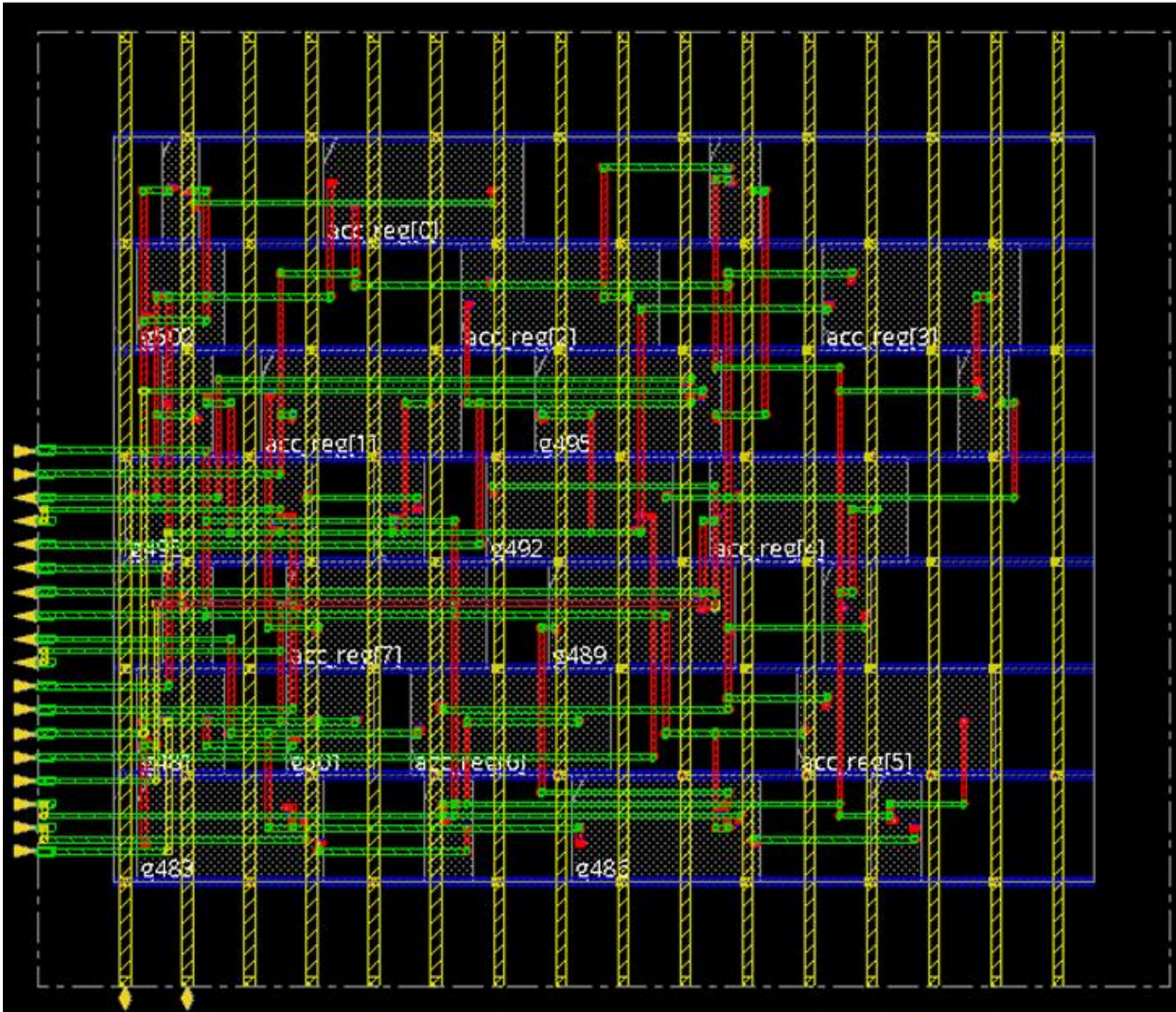
Unit: Micron

OK Apply Cancel Help

Powerplanning



Standard cell placement



CTS

```
create_route_type -name clkroute -bottom_preferred_layer Metal3 -top_preferred_layer Metal4
```

```
set_ccopt_property route_type clkroute -net_type trunk
```

```
set_ccopt_property route_type clkroute -net_type leaf
```

```
set_ccopt_property buffer_cells {CLKBUFX8 CLKBUFX12}
```

```
set_ccopt_property inverter_cells {CLKINVX8 CLKINVX12}
```

```
set_ccopt_property clock_gating_cells TLATNTSCA*
```

```
create_ccopt_clock_tree_spec -file ccopt.spec
```

```
source ccopt.spec
```

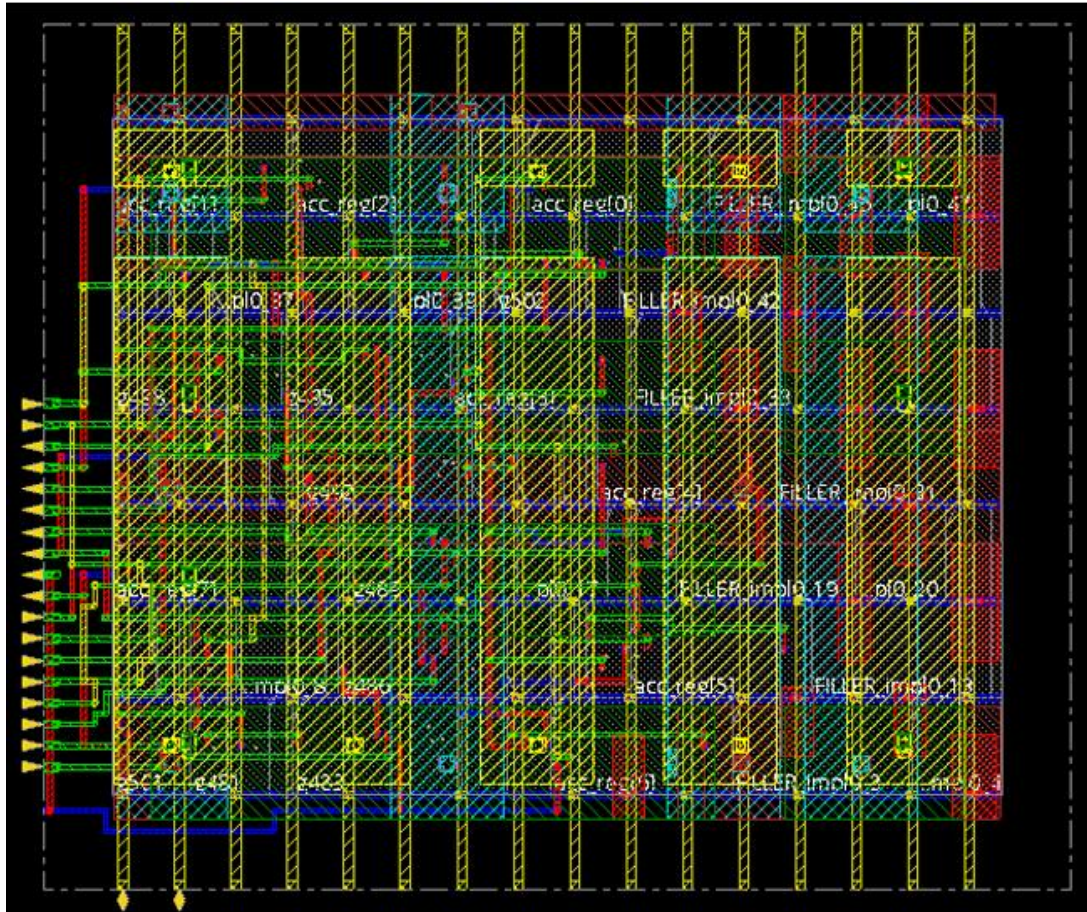
```
ccopt_design -cts
```

```
optDesign -postCTS
```

```
timeDesign -postCTS
```

```
saveDesign db/cts.enc
```

After routing and filler cell adding



DRC run

PVS 15.21-64b DRC Run Submission Form

File Preferences Tools Toolbars Help

cadence

Run Data

Rules

Input

Output

DRC Options

Start DRCDE ☒

Cancel

Apply

Submit

Rules

Tech&Rules Configurator

Technology

Technology mapping file /home/cad/VLSI2Lab/Analog/pvtech.lib

Technology gpd045_pvs

Rule Set default

Rules

Rules File(s)

Add

Remove

View

Edit

```
Total CPU Time           : 1(s)
Total Real Time          : 1(s)
Peak Memory Used         : 20 (M)
Total Original Geometry  : 1032(5050)
Total DRC RuleChecks     : 562
Total DRC Results        : 0 (0)
Summary can be found in file accu.sum
ASCII report database is /home/vlsi20/PnR/accu.drc_errors.ascii
Checking in all SoftShare licenses.
```

LVS+ERC

